

Product Requirements Document: VersaDocs

Version: 2.1 (Streamlined MVP)
Status: Approved for Development
Date: January 22, 2026
Privacy Tier: Confidential (PII Handling Required)
Project Leadership

Role	Assignee
Tech Lead / Architect	Dheyn Michael Orlanda (Necookie)
Frontend Lead	Jaypee Javier (Miquella)

1. Executive Summary

VersaDocs is a low-latency, AI-powered document generation engine designed to solve the "Blank Page Problem" for job seekers and professionals.

Unlike traditional text editors, VersaDocs employs a "Smart Wizard" Architecture. Users interact with a structured questionnaire to input raw, semantic data (e.g., "I fixed the login bug"). The system uses Groq (Llama-3) to instantly transform this input into professional narrative text, which is then deterministically rendered into an ATS-optimized PDF via @react-pdf/renderer.

Key Constraint: The application must be type-safe (TypeScript) and capable of rendering updates in <500ms.

2. Team Roles & Responsibilities

Dheyn Michael Orlanda (Necookie)
Role: Tech Lead / Architect
Domain: "The Data" (Backend & Infrastructure)

Key Responsibilities:

- System Architecture: Designing API Endpoints and Auth Logic.
- AI Integration: Prompt Engineering and Groq SDK implementation.
- Security: Rate Limiting and Data Sanitization.
- Data Layer: Drizzle ORM Setup and Database connection.

Jaypee Javier (Miquella)
Role: Frontend Lead

Domain: "The Glass" (UI/UX & Interaction)

Key Responsibilities:

- UI Implementation: Integration of shadcn/ui and Component Library construction.
 - Client State: Global Store management (Zustand) and Client-side Logic.
 - UX: Responsive Design, Accessibility (a11y), and Micro-interactions.
 - Validation: Client-side Zod schemas and Form error handling.
-

3. The Tech Stack (Frozen)

This stack is selected for Velocity and Type Safety.

Frontend (The Glass)

- Framework: Next.js 14 (App Router)
- Language: TypeScript (Strict Mode)
- Styling: Tailwind CSS + shadcn/ui
- State: Zustand (Global Store for Split-Screen Sync)
- Forms: React Hook Form + Zod

Backend (The Data)

- Database: Neon (Serverless PostgreSQL)
- ORM: Drizzle ORM
- Auth: Clerk (Middleware protected)
- AI Inference: Groq (Model: llama3-8b-8192)

Rendering

- Engine: @react-pdf/renderer (Client-side generation for speed)
-

4. System Architecture

4.1 High-Level Data Flow

1. Input: User types raw text into the Left Panel (Managed by Frontend Lead).
2. State Sync: Zustand store updates. The Right Panel (PDF) re-renders immediately for simple fields (Name, Phone).
3. Transformation: User clicks "Enhance". Backend API sends data to Groq.
4. Response: Groq returns structured JSON.
5. Update: The Store updates with the polished text. The PDF re-renders the new content.

4.2 Security Constraints

- Rate Limiting: Max 10 AI requests/minute per user (handled via Middleware).

- Data Isolation: All DB queries must use `where(eq(documents.userId, session.user.id))` to ensure users never see others' data.
-

5. Functional Specifications

5.1 The "Split-Screen" Workspace

- Layout: Resizable Panel Group (Left: 40% width, Right: 60% width).
- Left Panel (Form):
 - Must use `useFieldArray` for dynamic lists (Jobs, Schools).
 - Must support "Accordion" collapsing to save vertical space.
- Right Panel (Preview):
 - Must use `<PDFViewer>` from `@react-pdf/renderer`.
 - Performance: Must implement a 500ms debounce on typing to prevent PDF render lag.

5.2 The Transformation Engine

- Context Awareness: The API must accept a `targetRole` parameter.
 - *Example:* If `Target = "Manager"`, AI output uses words like "Led", "Orchestrated".
 - *Example:* If `Target = "Engineer"`, AI output uses words like "Built", "Optimized".
- JSON Enforcement: The Prompt must force the LLM to return valid JSON, not markdown text.

5.3 PDF Output Standards

- A4 Strict: The layout must be hard-coded to A4 dimensions.
 - Selectable Text: The output must be a true PDF text layer, not an image (ATS requirement).
 - Fonts: Fonts must be embedded to ensure consistent rendering across devices.
-

6. Definition of Done (DoD)

Before shipping to production, the following must be true:

1. Type Safety: `npm run type-check` passes with zero errors. (No any types allowed).
2. Responsiveness: The Left Panel (Form) is usable on Mobile devices (Right Panel/PDF is hidden on small screens).
3. Error Handling: If Groq/AI fails, the UI shows a friendly Toast notification, not a white screen.
4. Performance: PDF generation does not block the main thread for more than 100ms.